

# Application platform for AI workloads on Azure

You must carefully consider the application hosting platform that your AI workload is deployed on to ensure that you can maximize efficiency, the security of operations, and reliability.

This design area covers several types of applications that might be relevant to your AI workload:

- Exploratory data analysis (EDA)
- Model training and fine-tuning
- Inferencing

This article provides guidance for selecting the best platform for each of these functions to meet your business needs. There are also general recommendations that you can apply to all of these functions.

## Recommendations

Here's the summary of recommendations provided in this article.

 Expand table

| Recommendation                                                                             | Description                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reuse tools.                                                                               | Start by evaluating the tools you already use to understand whether they can be reused for your AI workload. If they support the required functionality and can meet your requirements for reliability, security, cost, and performance, bringing in a new tool might not be worth the cost and effort.                                                                    |
| Consider compliance requirements for your data and the regions that you plan to deploy in. | You might need to limit the regions you deploy in or isolate parts of your workload from each other to meet compliance requirements. Going into your designing phase with this information can help protect you from needing to redesign later.                                                                                                                            |
| Minimize building.                                                                         | Consider platform as a service (PaaS) or software as a service (SaaS) solutions to minimize the operational burden that building your own solution introduces, like patching and other maintenance. Minimizing the Day-2 burden required for the new technology simplifies your adoption. Many AI functions are complex, so we don't recommend building your own platform. |

| Recommendation                                            | Description                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Understand your quotas and limits.                        | When you design for the use of PaaS or SaaS solutions, understand any quotas or limits that apply. Your ability to scale out to meet high traffic demands might be affected by quotas or limits, so you might need to adjust your design to minimize that risk.                                                                                             |
| Deploy in the same region.                                | Try to deploy all related resources in the same region to reduce latency and to simplify the design.                                                                                                                                                                                                                                                        |
| Practice safe deployment.                                 | In general, you should treat the APIs for your AI workload the same as any other API in your environment. All APIs should be placed behind a gateway and all code should be handled with the same <a href="#">safe deployment practices</a> as every other code asset.                                                                                      |
| Establish performance benchmarks through experimentation. | Every AI workload is different, and the amount of compute that you need depends on your use case. Determine the amount and types of compute that are optimal for your workload by conducting thorough benchmark testing. This guide helps you choose a platform, but you'll only know which SKUs are appropriate for your workload after benchmark testing. |

## Considerations for the EDA platform

EDA is a common preliminary function that data scientists perform before modeling or statistical analysis. It can therefore be considered a development phase, which means that targets for reliability and performance might be significantly lower than those for production resources and maintaining productivity is the more important factor.

This section provides guidance on capabilities to consider when you select an EDA platform solution.

### Functional requirements

When you evaluate an EDA platform, consider the following questions:

- **Does the platform support transient usage?**

The platform should support transient workspaces and compute, which means that you should be able to stop the necessary resources when they aren't being used. This capability helps control costs. EDA jobs are typically interactive, so users need to be able to start VMs and stop them as they run jobs.

- **Does the platform support compute optionality?**

The platform should enable on-demand access to GPUs as needed and provide various compute options to help right-size the platform.

- **Does the platform support MLflow?**

Your EDA platform should make it possible to choose a technology that enables integration with MLflow for tracking your experiments. We recommend MLflow as a model development, deployment, and management protocol because it provides the following benefits:

- *Experiment tracking.* MLflow allows you to track experiments by recording parameters, metrics, and artifacts. This capability is essential during EDA so that you can keep track of different data preprocessing steps and feature engineering techniques and their impacts on model performance.
- *Reproducibility.* Because it logs all the details of your experiments, MLflow helps ensure that you can reproduce your results, which is critical for validating findings.
- *Data and model versioning.* MLflow helps with versioning datasets and models, which makes it easier to manage different versions of data transformations and tested models.
- *Collaborative work.* MLflow provides a centralized platform where data scientists can share their experiments and results, which facilitates collaboration and knowledge sharing.

## Nonfunctional requirements

Consider these questions as well:

- **How can the platform help control costs?**

The platform should enable data scientists to perform their work according to their schedule requirements, but it should be right-sized to ensure that cost expectations are met.

- **What security requirements must be followed for the platform?**

The data used during your EDA phase will probably be production data, which requires you to follow production practices to secure that data and monitor the platform. To that end, your platform should support all necessary security controls, including:

- *Access and authorization.*
- *Encryption at rest and in transit.*
- *Regional data protection requirements.*
- *Robust monitoring and alerting functionality, including logging and auditability.*

- *Private networking access to centralized repositories for container images, data, and code assets.*

## Tools

Use an [Azure Machine Learning compute instance](#) with team-level file shares as your EDA platform. One exception to this is if your team or organization is already using a suitable hosting platform, like GPU-enabled compute clusters in Databricks, for example. In that case, it might be more appropriate to stay on that platform.

### ⓘ Note

Don't build a full EDA platform unless you need to. GPU-optimized compute is expensive and isn't appropriate if your use case doesn't require it.

## Considerations for the model training and fine-tuning platform

When you move to model training and fine-tuning, you'll probably need high-performance GPU-optimized compute for the compute-intensive work that's required by those activities. Reliability typically isn't as important as performance because most of this work occurs behind the scenes. If high reliability is a requirement, evaluate whether spreading the workload across availability zones or regions is necessary. High reliability becomes more important when model freshness is updated frequently, which requires training to be completed on a tighter schedule. Your [RTO](#) should determine the reliability design that you choose.

The guidance in this section applies to both model training and fine-tuning. Unless you're forced to use separate platforms for these functions, you should use a single platform.

## Functional requirements

When you evaluate platforms for model training and fine-tuning, consider the following questions:

- **Does the platform support transient usage?**

Like EDA activities, model training and fine-tuning are typically not run full-time, so you should use a platform that can be stopped when it's not in use to help control costs. Unlike

EDA, however, model training is typically a batch process, so the compute is only needed when the batch runs and can be shut down until the next run.

- **Does the platform provide orchestration?**

Because of the complexity required in managing the compute for model training and fine-tuning, we recommend an orchestrator.

- **Can existing technologies in your environment be part of the solution?**

If your existing data platform has machine learning capabilities, like [Azure Databricks](#) does, you can use it for certain steps, like data transformation and feature engineering, training, fine-tuning, and other steps in Machine Learning. Combining technologies can help you minimize the cost and complexity involved in using a data platform for functions it might not be ideally suited for.

## Nonfunctional requirements

Consider this question as well:

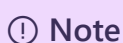
- **What's the tolerable tradeoff between cost and performance?**

Given the high-performance, GPU-optimized compute requirements, ensure that you test and benchmark your training and fine-tuning extensively to determine the ideal SKU that balances performance against costs.

## Tools

We recommend Azure Machine Learning for the model training and fine-tuning platform because it provides orchestration functionality with support for batch compute. There are two compute options to evaluate:

- [Serverless compute](#) is ideal for short, infrequent runs that can tolerate noisy neighbor effects. You can choose either standard pricing or spot pricing. Spot pricing is only recommended for highly interruptible training. Don't use serverless compute for full-time operations. The costs can escalate quickly.
- [Compute clusters](#) give you significant control over available hardware and are tuned for parallel or distributed training.



For foundation models, your choice of model hosting platform might limit your fine-tuning options. For example, using Azure OpenAI Service for model hosting limits your fine-tuning options to the built in Azure OpenAI fine-tuning functionality.

## Considerations for the model hosting and inferencing platform

Model hosting and inferencing functions make up the serve layer of the AI workload. These functions are performed with endpoints that are specific to the software that you use. Model-serving software solutions, like NVIDIA Triton, TorchServe, and TensorFlow Serving, are essentially Python SDKs that front a model with an API and add functionality that's specific to the solution. You can choose your hosting platform based on your choice of software or choose your software based on your choice of hosting platform.

When you use SaaS or PaaS solutions with prepackaged models, like the large language models that are available in Azure OpenAI, you have few or no opportunities to select a serving software. Instead, the service that you're consuming provides an API. This reduces flexibility in the process for creating a model deployment, which can provide advantages and disadvantages. For example, it can streamline the development process of your workload. On the other hand, it reduces flexibility in how your application can call and interact with the model.

Fundamentally, the APIs for the serve layer are microservices, so you should follow the same practices for these APIs that you follow for other microservices in your environment. They should be containerized, [bulkheaded](#) from other services, and have their own lifecycles that are independent of other services and APIs. Keep in mind, however, that serve layer APIs generally require significantly more GPU-based compute power and larger container images than traditional APIs.

This section provides guidance on capabilities to consider when you select a model hosting and inferencing platform.

### Functional requirements

When you evaluate platforms for model hosting and inferencing, consider these questions:

- **Does your workload require batch or online inferencing?**

The inferencing endpoints are used either for batch or online inferencing processes, and the inferencing method helps determine the right hosting platform. Batch inferencing is best

hosted on a platform that supports transient usage and allows the compute to be shut down when it's not being used. Online inferencing is best hosted on a platform that supports elastic compute utilization, which scales automatically based on the load at any given time.

- **Does the platform support traceability?**

Traceability is critical for maintaining the integrity of the models used in your workload. It's important to know information about the model, like the current version, who deployed it, when it was deployed, and the model's data lineage.

Apply meaningful tags to images in your container registry to ensure that your model hosting service pulls a specific version that the team can easily identify. This approach helps with data governance by reducing the risk of outdated or incorrect models being used in production.

- **Will your hosting platform be a centralized resource?**

Many organizations use a centralized model hosting platform that's used by different teams for their own workloads. If your hosting platform is centralized, you should consider whether you need support for chargeback. This functionality allows you to track platform utilization by team and workload.

## Nonfunctional requirements

Consider these questions as well:

- **What are the reliability requirements for the platform?**

Serve layer APIs are production resources, so you should apply the same reliability requirements to them that you apply to other workload flows that match their [criticality](#) rating. If their criticality requires high availability, your hosting platform should support availability zones or a multiregion design.

- **What networking controls are required for the platform?**

Determine whether you need private networking or an egress firewall to provide protection for the platform.

- **What are the identity and access security requirements for the platform?**

Determine the identity and access controls that are required for your endpoints. Consider whether you need native role-based access control (RBAC) or built-in support for your identity and access platform, for example, Microsoft Entra ID.

- **What monitoring capabilities does the platform support?**

Determine the required monitoring capabilities for your endpoints. Depending on the platform, you might have limited access to logs and metrics, which might limit your ability to audit activities or detect malfunctions.

- **What are the performance requirements for the platform?**

Inference latency is a common concern, and different platforms have different performance profiles. Serverless and PaaS services that use a utility model can be affected by the noisy neighbor problem and often have no throughput guarantees. On the other hand, the same platforms might offer a self-hosted option that provides guaranteed throughput with a pre-purchasing model. You can also consider self-hosting on Kubernetes for more predictable latency behavior.

Be aware of service limits and quotas that might affect your performance, like those for [Azure OpenAI](#). Often these quotas and limits are aggressively set to meet capacity demands, so if your choice of platform doesn't provide the performance that you require, you might need to adopt strategies to spread the compute demand across instances.

Advanced architectures can combine multiple deployments to achieve fixed throughput for the bulk of the workload and bursting capabilities for a more flexible compute.

## Tools

### Batch inferencing

- If you're performing inferencing on data that resides in a platform that supports model hosting, like Databricks, consider using that platform for inferencing. Be sure to isolate the inferencing compute from other functions performed by the data platform.
- We recommend [Azure OpenAI Batch API](#) for foundation models.
- For non-foundation models, consider these recommendations:
  - Consider using [Azure Machine Learning batch endpoints](#) for the following scenarios:



- You need to perform inferencing on a large dataset that's distributed in multiple files and you don't require low latency.
- You need to perform long-running batch operations over large datasets and can take advantage of parallelization.
- You need to deploy pipeline components for batch processing.
- If you need to run Spark jobs for distributed data processing, consider using [Azure Synapse Analytics](#), [Databricks](#), or [Machine Learning serverless Spark compute](#).
- If none of these scenarios apply, we recommend Machine Learning batch endpoints.

## Online inferencing

- Evaluate platform PaaS and serverless solutions as a first step. These services are typically the easiest to adopt and manage because they simplify your design and minimize operational burden. For example, Azure OpenAI is a good choice for foundation models.
  - Consider using the Azure Machine Learning Serverless API to aggregate endpoint access even if you use Azure OpenAI or another foundation model hosting solution.
- Consider Machine Learning with managed compute clusters when PaaS or serverless solutions aren't the best fit. Compute that's managed by Machine Learning supports traffic splitting and mirroring for A/B testing, debugging, and robust auditing. Because the compute is managed by the service, Day-2 operations are easier when you self-host your model. Managed compute also offers a wide selection of compute configurations and scaling capabilities.
- If you choose to self-host your model on an Azure Kubernetes Service (AKS) cluster that's [attached to Machine Learning](#) or another container-based platform, be sure that the node pool is isolated from other APIs or any other workloads on the cluster to achieve predictable performance and to optimize security. Avoid using GPU-based or GPU-optimized compute for anything other than your AI workload functions to reduce costs. Instead, establish your performance baseline through testing and right-size your compute to meet your performance requirements without over-provisioning.
- You can also self-host your model by using infrastructure as a service (IaaS) solutions, like [Azure Data Science Virtual Machine](#).

## Considerations for the orchestration platform

Orchestration, in the context of AI workload application platforms, refers to tools like prompt flow in [Machine Learning](#) and [Microsoft Foundry portal](#). These tools are designed to streamline the entire development cycle of AI applications by automating many common workflow functions.

## Nonfunctional requirements

As with all other production workloads in your cloud estate, when you evaluate orchestration tools, you need to consider:

- **Reliability, security, and monitoring.** Orchestration tools should adhere to reliability, security, and monitoring standards for production workloads.
- **Performance.** Orchestration tools don't require GPU-optimized or GPU-based compute, consider general-purpose SKUs.
- **Cost optimization.** Orchestration tools are *always-on*, consider elastic compute options to minimize utilization costs.

## Tools

- Prefer an off-the-shelf solution like prompt flow. Determine whether its capabilities match your orchestration needs before you look into custom hosting with tools like LangChain or Semantic Kernel.
- Host endpoints for solutions like prompt flow on Machine Learning with compute instances or on AKS with self-hosting.

## Next steps

Design area: Training data design

Last updated on 11/18/2024